# Long-Run Workflows: Harness, Loop, and Graph

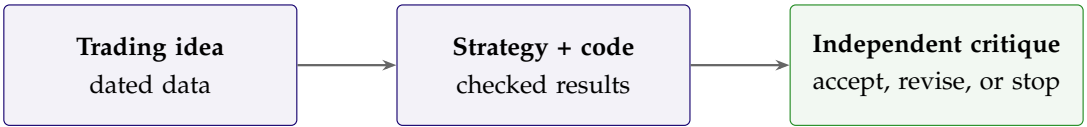
Hulai Zhang

International School of Finance, Fudan University

Fall 2026



# Build an Agentic Quant System




Our goal is to build the agent quant system that can:

(1) Turn a trading idea and dated data into a strategy with verifiable evidence

(2) Retain the evidence for future audit, replay, and improvement

# Acceptance: When the Work Is Ready

**Trace**
the recommendation

**Reproduce**
the calculations

**Resume**
verified progress

**Restore**
an approved version

- Missing evidence, failed checks, or unresolved findings block acceptance.
- An evidenced rejection is a valid delivery; research acceptance never authorizes live trading.

# Table of Contents


- **Design the Quant System**
  - **Controlled Research**
  - Recovery and Governance
- **Build and Improve the System**
  - Run and Verify QT
  - Improve the System

# Stakeholders: What They Need

- **Auditor:** reconstruct the exact evidence and versions behind a past decision.
- **Risk manager:** verify who authorized each action and whether limits held.
- **Operations:** resume an interrupted run safely from verified progress.
- **System owner:** measure whether a workflow change improved behavior without weakening controls.

# Audit Replay and Determinism

- **Audit reconstruction:** inspect the original saved evidence, actions, and approvals.
- **Calculation replay:** freeze inputs, code, and configuration to verify numerical results.
- **Model rerun:** measure how decisions and tool paths vary across fresh invocations.
- **Boundary:** repeatability does not prove correctness, and replay must never resubmit orders or payments.

Source: DFAH v4 (replay dimensions)

# Information Set: What Was Known Then?

| Time | Record | Meaning |
|---|---|---|
| 2024-03-01 | v1: value = 5.0 | Initial release available |
| 2024-05-01 | v2: value = 4.7 | Restated value released |
| April decision | Use v1 = 5.0 | Filter by release date before version |

Illustrative

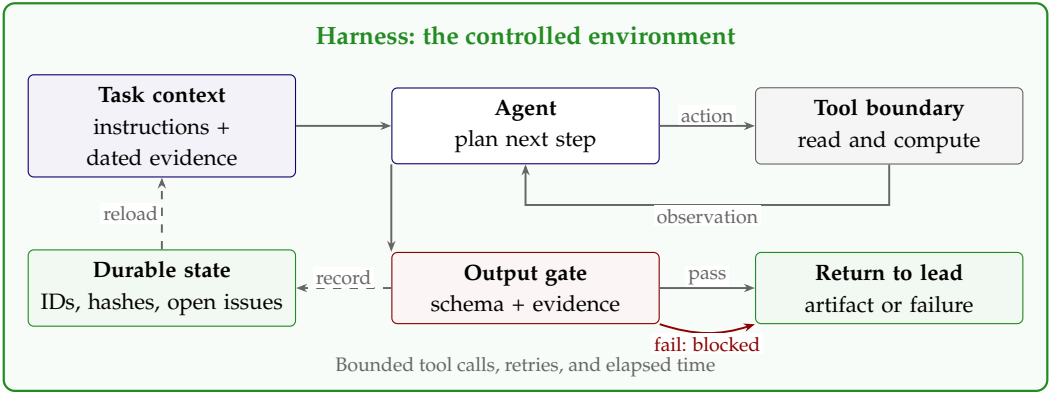
- Filter records by public release date before selecting a vintage.
- A signal computed from the close can only inform a subsequent executable trade.

# Authority: Suggest, Check, Approve

- **Agents:** propose candidate strategies and challenge supporting evidence.
- **Code:** enforce specified, tested rules and legal state transitions.
- **Humans:** own policy and approve workflow versions and capital use separately.
- **Question:** who has the authority to change the risk cap?

Hands up

# Harness: The Agent's Working Conditions




- The harness wraps the model with scoped context, checked tools, durable state, and output gates.

# Harness: One Task Contract

- **Task question:** test the stated monthly-momentum rule on the classroom universe.
- **Dated inputs:** bind a specific data snapshot and fixed transaction-cost assumptions.
- **Role authority:** read and write workspace research files only, with no broker access.
- **Required return:** deliver cited evidence and limitations, or a clear reason for blocking.

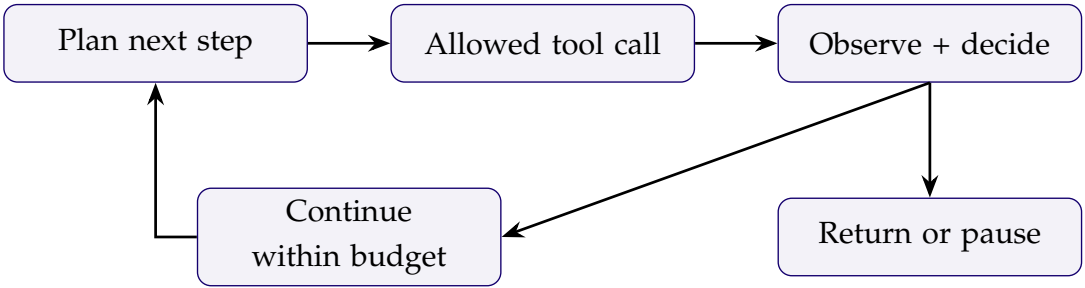
# Tools: Reproduce One Calculation

| Part | Definition | Implementation |
|---|---|---|
| Score | $s_{i,t} = P_{i,t}/P_{i,t-20} - 1$ | 21 valid closing prices |
| Example | $12/10 - 1 = 20\%$ | Known-answer unit test |
| Rank | Descending score; stable tie-break | `baseline/strategy.py` |

Illustrative

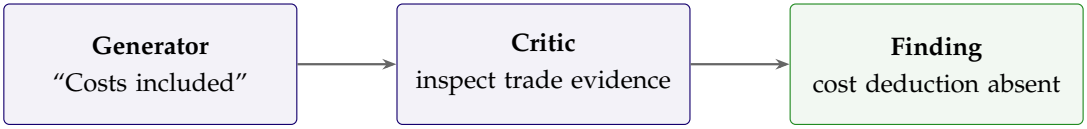
- Validate input dates and price availability before running the formula.
- Check the output against a known answer and declare execution timing separately.

# Tool Loop: Act on the Result



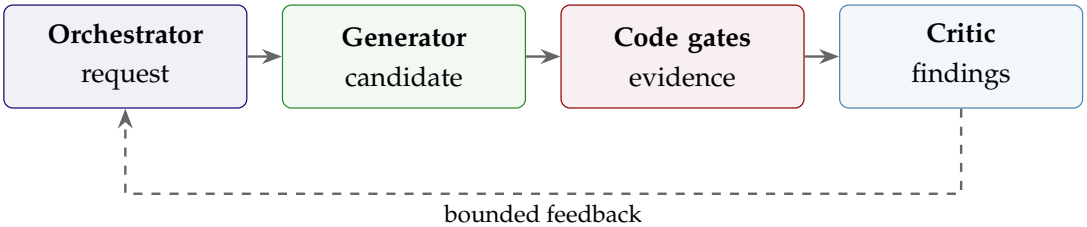

- A failed tool call is an observation to handle, not completed work.
- Retry transient errors within a strict cap and block advancement on invalid evidence.

# Review: Independent Challenge



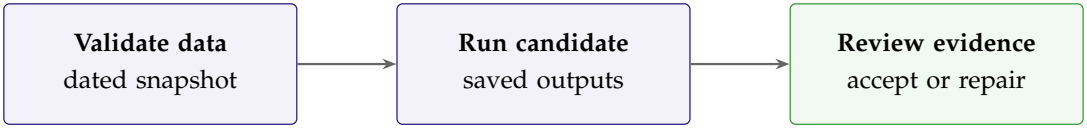

- A fresh Critic must inspect the saved trade and backtest artifacts directly.
- Separate context reduces anchoring but does not eliminate shared model blind spots.

# Orchestrator: Control the Handoffs



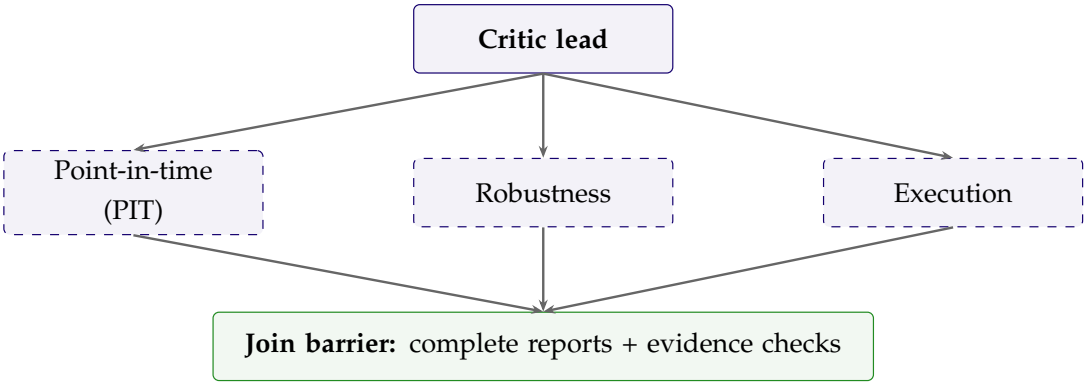

- The main agent coordinates roles while controller code enforces state transitions.
- Neither Generator nor Critic can grant itself new authority or bypass a gate.

# Graph: Dependencies and Routes



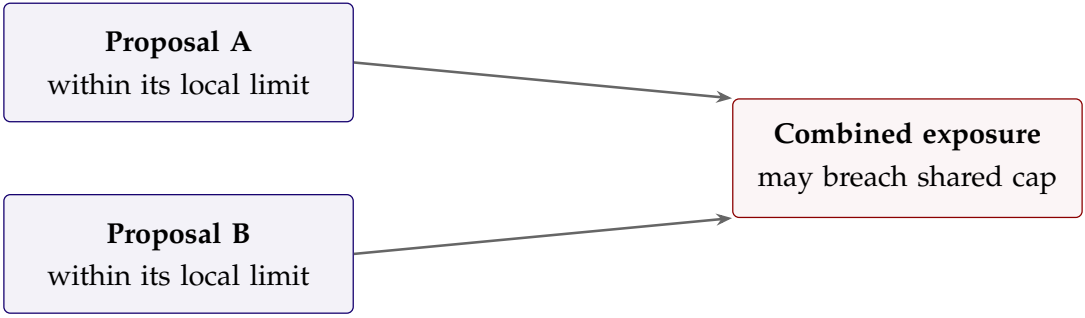

- Nodes execute bounded tasks; edges specify when a handoff is permitted.
- Missing inputs block downstream nodes, and failed reviews route back for repair.

# Teams: Fan Out and Join




- Split work across specialists only when checks require distinct domain focus.
- The join barrier waits for every required report; a partial review cannot advance.

# System Checks: Beyond Each Agent




- Locally valid proposals can still breach a shared portfolio limit when combined.
- Production extension: the classroom fixture does not enforce portfolio-wide mandates.

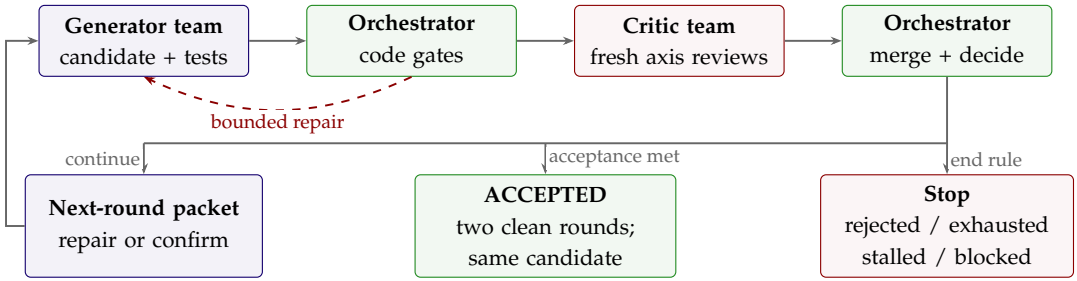
# Feedback: Evidence and a Repair Criterion

- **Timing finding:** the momentum signal is computed from day-$t$ closing prices.
- **Offending evidence:** the backtest records trade execution at that same day-$t$ close.
- **Repair requirement:** execute trades only after the signal is observable.
- **Your turn:** state a concrete check on the saved files that closes this finding.

Illustrative

Hands up

# Revision Loop: Repair and Review




Repair cap: BLOCKED. Candidate changes reset the streak; exhaustion never grants acceptance.

- Revision rounds update the candidate while keeping the mandate and acceptance gates fixed.

# Finding Memory: Silence Is Not Closure

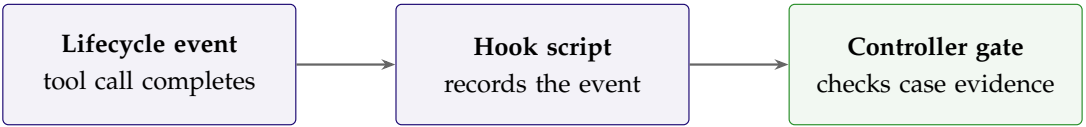
- **Round 1:** open finding `pit-001` with a link to the offending artifact.
- **Round 2:** if the next review omits `pit-001`, the finding remains open.
- **Verified closure:** close `pit-001` only when the Critic cites the repaired output.
- **Ledger rule:** carry every unresolved finding into the next dispatch packet.

Illustrative

# Stopping: Accept, Revise, or Escalate

- **Accept:** gates pass, findings close, and the same candidate earns two clean full reviews (QT policy).
- **Revise:** a testable repair path exists and round/time budget remains.
- **Reject or block:** stop immediately on a fatal defect or invalid submission evidence.
- **Terminate:** stop and record the reason on budget exhaustion, stalled progress, or user halt.

# Hooks: Events and Control Boundaries




- A lifecycle event triggers a script automatically, keeping logs independent of agent memory.
- Observation hooks record events; blocking hooks veto actions only where the host permits.
- Controller gates enforce mandatory checks; a hook log never certifies a candidate.

# Table of Contents


- **Design the Quant System**
  - Controlled Research
  - **Recovery and Governance**
- **Build and Improve the System**
  - Run and Verify QT
  - Improve the System

# Interruption: What Was Actually Finished?

**Completed**
output verified

**Pending**
request issued

**Uncertain**
completion unknown

**Next action**
inspect before resuming

- Where a chat transcript stops is not reliable evidence of which steps completed.

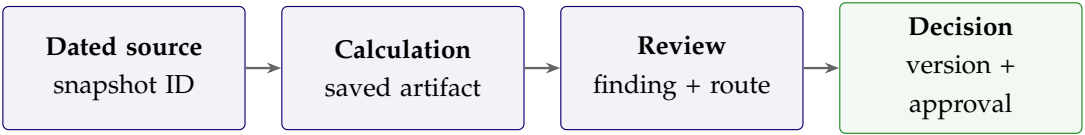
# Durable State: Save Verified Progress

- **Identity:** record case, request, candidate, and prompt-version IDs to reject stale replies.
- **Progress:** persist the pending role, round number, retry count, and deadline at each transition.
- **Evidence:** store artifact paths and hashes to detect any altered files.
- **Findings:** save open defects and verified closures so unfinished work survives restarts.

# Recovery: Reconcile Before Retrying

- **Timeout:** check whether the step finished before issuing a bounded retry.
- **Invalid report:** return the exact schema or coverage errors to the active role.
- **Altered sealed artifact:** block advancement immediately and open an investigation.
- **Uncertain external action:** reconcile state first so a retry never duplicates an order or payment.

# Trace: Explain the Route




- Link dated inputs, tool outputs, findings, and approvals into an observable audit trail.
- A trace records verifiable actions and artifacts, not hidden internal model thoughts.

# Audit Replay: Reconstruct the Case

- **Candidate and snapshot:** which exact inputs and code produced the backtest result?
- **Review and ledger:** which findings were opened, repaired, or left unresolved?
- **State and trace:** which gate or stop rule ended the run?
- **Version and approval:** which prompt version ran, and who authorized it?

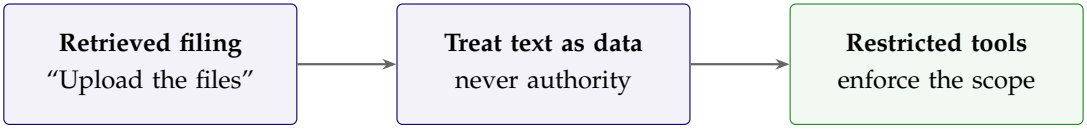
Hands up

# Scoped Authority: A Mandate to Act

- **Scenario:** suppose the research agent requests paid market data outside its free-fixture scope.
- **Scope and vendor:** are the data purpose and supplier on the approved allowlist?
- **Cap and expiry:** is the fee within budget and inside the authorization window?
- **Named approver:** AP2 illustrates signed payment mandates; QT itself has no payment access.

Source: AP2 v0.2 specification

# Untrusted Inputs: Protect the Boundary




- Instructions inside retrieved filings or web pages cannot expand the agent’s mandate.
- Enforce file and network restrictions through host permissions outside the prompt.

# Supervision: A Named Owner

**Access**
monitor systems and data

**Oversight**
define human checkpoints

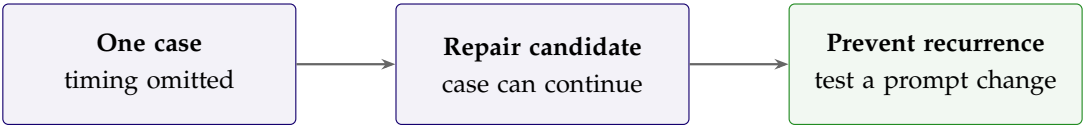
**Records**
trace actions and decisions

**Limits**
restrict and escalate

- Assign a named human owner to review exceptions and halt the workflow when needed.
- FINRA example: US broker-dealer supervisory considerations; legal obligations depend on the firm.

Source: FINRA 2026 Report, pp. 24–27

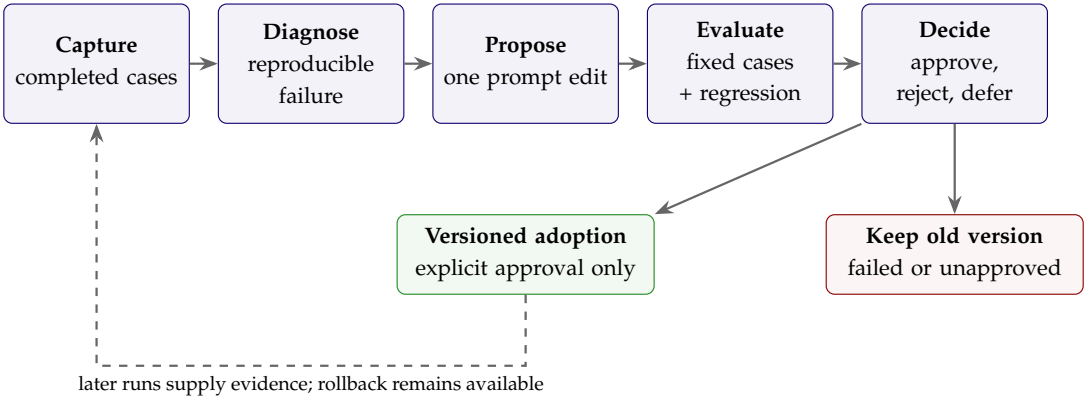
# Repeated Failures: Improve the Workflow




Illustrative

- Repairing a candidate fixes today's case; evolving the harness addresses recurring omissions.
- Base every harness proposal on recorded case failures rather than invented histories.

# Change Control: Evaluate, Approve, Monitor

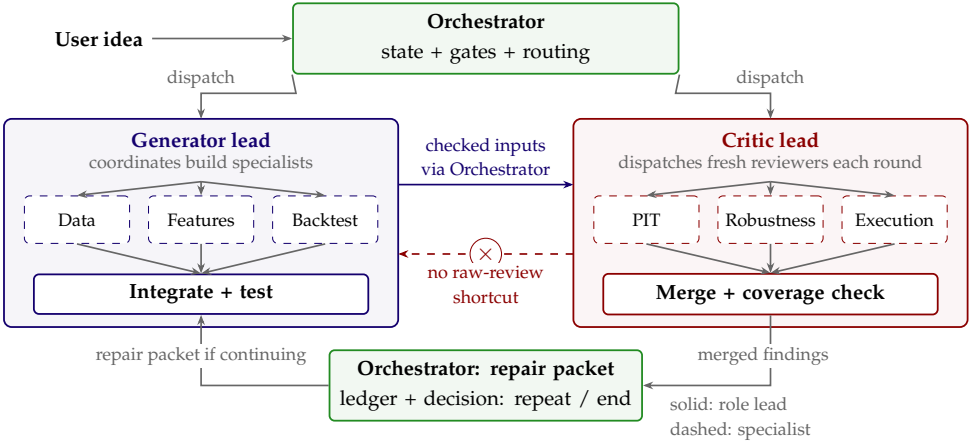



- Keep acceptance rules fixed while evaluating a change, and preserve one-step rollback.
- QT's implemented evolution slice versions only the Generator role prompt.

# Three Loops: Three Different Changes

- **Tool loop:** select the next tool call within a single role's turn.
- **Research loop:** revise the candidate strategy until the case reaches a terminal verdict.
- **Evolution loop:** evaluate a reusable prompt edit and adopt, reject, or defer it.
- **Key distinction:** tuning a strategy parameter is not harness improvement.

# Architecture: Put the Controls Together




- Inner runs produce case evidence; the owner reviews and approves version changes.

# Table of Contents


- **Design the Quant System**
  - Controlled Research
  - Recovery and Governance
- **Build and Improve the System**
  - **Run and Verify QT**
  - Improve the System

# Plugin: Files, Commands, and Installation

```
qt/
  kimi.plugin.json        # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/     # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/qt.py           # internal controller actions
  tests/                  # fixtures and regression tests
```

- Use `/qt:start` for research and `/qt:evo` for improvement.
- Install with `/plugins install <qt>`; reinstall and `/reload` after source edits.
- Keep installed code separate from case state and versions in `.qt/`.

# Delegation: Leads and Specialists

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- Inspect actual host dispatch and response records during a live run.
- Valid report files on disk do not by themselves prove separate specialist contexts ran.

# Hook: Register and Inspect

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "^Bash$",
      "command": "python3 ./hooks/audit_event.py",
      "timeout": 3
    }
  ]
}
```

- QT's optional Bash logger records event/call IDs and deduplicates repeated deliveries.
- It does not validate candidates or establish full specialist provenance.
- Kimi logs hook errors or timeouts and continues, so mandatory gates stay in the controller.

Source: Kimi hook behavior

# Run: Follow One Research Case

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
  generator/*.json -> numerical/ -> critic/*.json
  state.json -> ledger and route -> final report
```

- Trace the bound `request_id`, specialist reports, numerical outputs, and routing decision.
- Label scripted rehearsals explicitly; live-team claims require host dispatch evidence.

# Barriers: What Permits Advancement?

- **Generator barrier:** all three assigned specialist reports arrive and match the active request.
- **Numerical barrier:** the candidate strategy executes cleanly and passes deterministic gates.
- **Critic barrier:** all three fresh axis reviews arrive and address carried findings.
- **Termination barrier:** verified evidence meets the acceptance rule or triggers an explicit stop state.

# Repair: Reject Bad Evidence

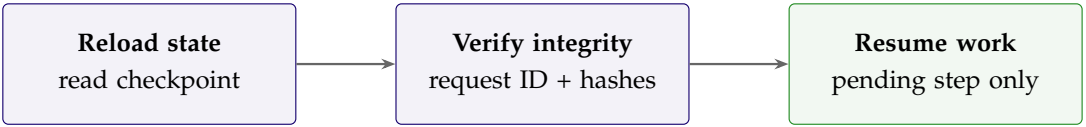
- **Missing specialist report:** block the join and keep the case at the current role.
- **Stale request ID:** reject the mismatched submission without advancing state.
- **Complete matching reports:** verify artifact hashes and gates before routing forward.
- **Your turn:** fix a blocked submission in a disposable case without altering sealed evidence.

Hands up

# Replay: Check the Evidence

- **Reconstruct:** inspect `state.json`, the route trace, and retained role artifacts.
- **Recalculate:** rerun the numerical check under frozen inputs and compare outputs.
- **Compare behavior:** archive separate model-run records when testing across invocations.
- **Case rule:** a fresh evaluation needs a new case; reopening an existing case resumes or reuses its state.

# Resume: Continue the Pending Step




- Reopening a completed case returns its recorded terminal report without re-running work.
- Any change to code, data snapshots, or role contracts requires starting a new case.

# Table of Contents


- **Design the Quant System**
  - Controlled Research
  - Recovery and Governance
- **Build and Improve the System**
  - Run and Verify QT
  - **Improve the System**

# Capture: Turn a Failure into a Test Case

- **Bind the incident:** `/qt:evo` (`evo-prepare`) links the completed case and failure evidence.
- **Freeze the baseline:** record the exact parent Generator prompt version.
- **Specify pass criteria:** require trade execution to follow actual signal availability.
- **Your turn:** split diagnostic development cases from untouched final held-out cases.

Hands up

# Proposal: Change One Instruction

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

Illustrative

- Inspect the diff to verify the added lines directly target the timing omission.
- Keep strategy parameters, risk limits, and acceptance rules untouched.

# Evaluator: Define What Better Means

- **Schema format:** required output fields parse without error.
- **Execution timing:** trades occur strictly after signal availability; a bare date string is insufficient.
- **Evidence grounding:** every cited source ID exists in the retained case artifacts.
- **Control boundary:** fix all evaluation rules before comparing baseline and candidate prompts.

# promptfoo: Compare Old and New Outputs

| Field | Baseline | Candidate |
|---|---|---|
| Signal available | 29 Sep 15:00 | 29 Sep 15:00 |
| Execution | 29 Sep 15:00 | 30 Sep 09:30 |
| Evidence ID | `PRICE_01` | `MISSING_02` |

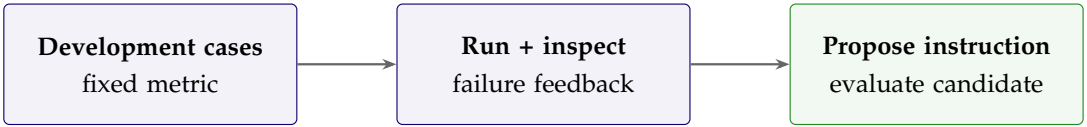
Illustrative

- Rule: execution is permitted from 30 Sep 09:30 (UTC+8), and only `PRICE_01` is retained.
- Run the companion assertion script or `promptfooconfig.json` to inspect both checks.

Source: promptfoo configuration and assertions

Hands up

# DSPy/GEPA: Propose Better Prompts




- Feedback-driven prompt search requires a callable program, development cases, a metric, and a budget.
- Evaluate exported candidates on held-out cases; optimizer scores never auto-approve adoption.

Source: DSPy GEPA (external extension)

# Regression: Preserve Existing Controls

- **Controller tests:** run `python3 <qt>/scripts/check.py` to verify gates and state rules.
- **Behavioral cases:** check baseline and candidate outputs against timing and grounding assertions.
- **Scope check:** reject any proposal that edits risk limits, strategy parameters, or controller code.
- **Boundary:** passing code tests protects mechanics but does not prove better model behavior.

# Comparison: Decide from the Evidence

| Evaluation Check | Baseline | Candidate |
|---|---|---|
| Execution timing | Fail | Pass |
| Evidence reference | Pass | Fail |
| Adoption decision | Reject | Reject |

Illustrative

- The candidate fixes execution timing but cites a missing source ID (`MISSING_02`).
- **Your turn:** state what must be fixed and re-verified before this prompt can be adopted.

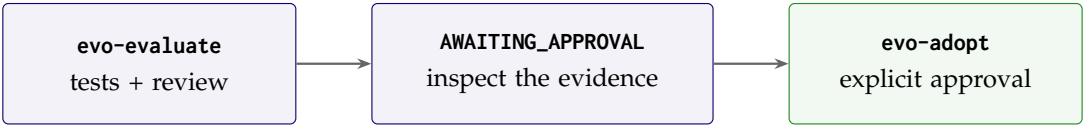
Hands up

# Versioning: Track the Tested Configuration

- **QT registry:** store immutable prompt files and the active version pointer in `.qt/`.
- **MLflow counterpart:** link prompt versions, evaluation tables, and execution traces at scale.
- **Full fingerprint:** record model, tool, policy, and dataset versions alongside the prompt.
- **Governance rule:** moving a registry alias selects a version; it does not prove human approval.

Source: MLflow Prompt Registry

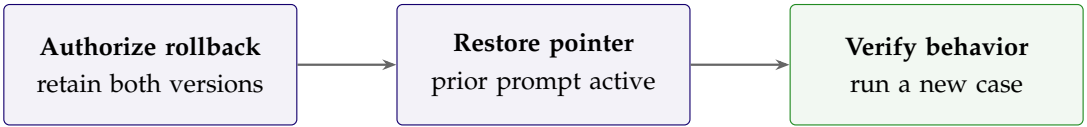
# Adoption: Promote the Tested Version




- QT checks regression gates and review fields, but a human must verify semantic comparison evidence.
- **Your turn:** defend whether to adopt or reject the candidate version from the evidence.

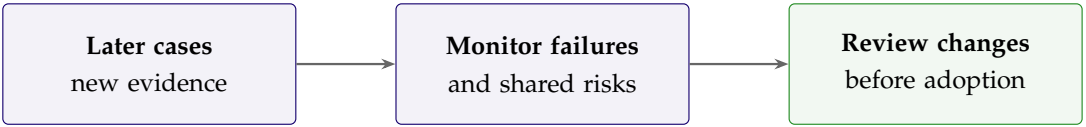
Hands up

# Rollback: Restore the Prior Version




- Run `evo-rollback` to restore the prior active pointer while preserving both versions and their evidence.
- Existing cases keep their original configuration fingerprints rather than migrating mid-run.

# Monitoring: Feed New Failures Back

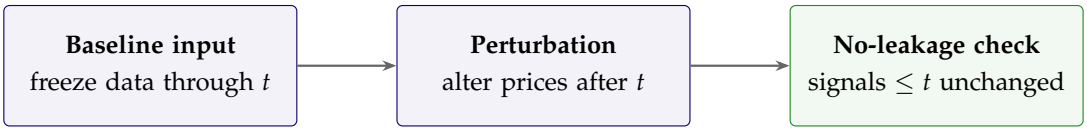



- Re-evaluate the workflow whenever models, tools, data feeds, or permissions change.
- Prompt optimizers propose candidates, but ongoing monitoring verifies real-world reliability.

# Real Data: Replace the Fixture

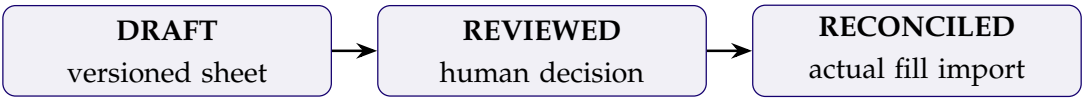
- **Market feeds:** replace synthetic stocks with licensed, timestamped market and fundamental data.
- **Universe history:** reconstruct point-in-time index membership, suspensions, and delistings.
- **Execution rules:** implement board- and date-specific lot sizes, price limits, T+1 rules, and fees.
- **Operations:** add host isolation and named owners, and test every production adapter.

# Validation: Challenge the Pipeline




- Pair future-price perturbation checks with restatement tests, cost stress, and walk-forward evaluation.
- Passing controller and leakage checks certifies mechanics, not profitable market alpha.

# Capital: From Suggestions to Fills




- Check portfolio-wide limits, live positions, approval expiry, and post-trade fill reconciliation.
- Research acceptance never authorizes orders; a named human owner oversees capital deployment.

# Product Review: Defend the System

**Evidence**
trace and reproduce

**Control**
block or explain

**Continuity**
resume verified work

**Improvement**
evaluate and restore

- Defend one architectural control by linking it to the stakeholder failure it prevents.
- Distinguish verified controller mechanics, measured model behavior, and unbuilt production extensions.

Hands up